

warehouse-linker

개념 정리 학습 노트

도매 식자재 재고관리 · 수발주 · 배차관리 프로그램을 만들며 사용한
눈에 보이지 않는 설계·로직 개념과 개발 전체 과정

1부. 개발 전체 과정: 요구사항에서 운영까지

1. 개발 단계 한눈에 보기 · 2. 단계별 개념 · 3. 요청이 처리되는 흐름 · 4. 코드 구조

2부. 핵심 개념

1. 데이터 모델링 · 2. 데이터 정합성 · 3. 동시성 · 4. 업무 흐름 모델링

5. 보안 · 6. 오류 처리 · 7. 아키텍처와 성능 · 8. Next.js·프론트엔드

9. UI/UX 디자인 시스템 · 10. 개발 도구와 운영

3부. 공부 순서 추천과 용어 색인

작성일 2026-10-03 · 기준 문서 `docs/DESIGN.md`, `CLAUDE.md`

기술 스택: Next.js 16 (App Router) · TypeScript · Tailwind CSS 4 · shadcn/ui · Prisma 6 · PostgreSQL 15 ·
Docker

1부. 개발 전체 과정

1. 개발 단계 한눈에 보기

① 요구사항
무엇을 만드나

② 설계
어떻게 만드나

③ 환경 구성
어디서 만드나

④ 구현
만든다

⑤ 검증
제대로 됐나

⑥ 배포·운영
쓰게 한다

⑦ 유지보수
고쳐 나간다

실제 개발은 이 단계를 한 번에 끝내지 않고 기능 하나씩 작게 반복합니다(반복·점진적 개발). 이 프로젝트도 상품 → 입고 → 출고 → 재고현황 → 창고 → 수발주 → 배차 → 권한 → 감사 로그 → 칸별 재고 순으로 기능을 늘려 왔고, 그때마다 설계 문서를 함께 고쳤습니다.

아래 표시: **적용** 이 프로젝트에 이미 있음 · **부분** 일부만 있음 · **미적용** 앞으로 필요

2. 단계별 개념

① 요구사항 정의 Requirements

하는 일 누가(사용자·역할), 무엇을(기능), 어떤 규칙으로(업무 규칙) 쓰는지 정리합니다.

관련 개념 기능 요구사항(입고 등록, 배차 배정 등) / 비기능 요구사항(속도, 보안, 동시 사용, 시간대) / 업무 규칙(재고는 음수 불가, 직원 등록은 관리자가 확정) / 사용자 역할(직원·관리자) / 범위(Scope)와 우선순위(지금 할 것과 나중에 할 것: DESIGN.md의 "미결 / 추천 후보")

이 프로젝트 **적용** DESIGN.md에 업무 규칙과 불변식, 미결 항목을 기록

② 설계 Design

하는 일 데이터를 어떻게 저장할지(데이터 모델), 코드를 어떻게 나눌지(아키텍처), 화면을 어떻게 보일지(UI 설계)를 정합니다.

관련 개념 데이터 모델링 / ERD(엔티티와 관계) · 불변식 정의 · 상태 전이 설계 · 권한표 설계 · 아키텍처 결정(모듈러 모놀리스) · 디자인 시스템 · 설계 문서화(결정한 이유까지 남기기)

이 프로젝트 **적용** docs/DESIGN.md (업무 규칙·모델), docs/WMS_design-system_indigo.md (UI 규칙), CLAUDE.md (개발 원칙)

③ 개발 환경 구성 Environment Setup

하는 일 누구 컴퓨터에서든 같은 방법으로 실행되게 만듭니다.

관련 개념 패키지 관리(package.json, package-lock.json로 버전 고정) · 컨테이너(Docker로 DB 실행) · 환경 변수(DB 주소·비밀값을 코드 밖에 둠) · 마이그레이션(DB 구조를 코드로 관리) · 시드(초기 데이터) · 환경 분리(개발 / 운영: 운영에서는 쿠키에 secure 적용)

이 프로젝트 **적용** docker compose up -d → npx prisma migrate dev → npx prisma db seed → npm run dev

④ 구현 Implementation

하는 일 설계에 맞춰 코드를 작성합니다. 정해진 순서가 있으면 실수가 줄어듭니다.

권장 순서 스키마 변경 → 마이그레이션(필요한 CHECK 제약 추가) → validation.ts(입력 검증) → service.ts(트랜잭션과 DB) → actions.ts(권한 확인 후 서비스 호출) → codes.ts(라벨·상수) → 화면(app/<screen>/) → 감사 로그 추가 → 문서 갱신

관련 개념 관심사 분리 · 단일 책임 · 재사용 · 코딩 규칙(컨벤션) · 작은 단위로 커밋

⑤ 검증 Verification / Testing

하는 일 만든 것이 요구사항대로 동작하고 기존 기능을 망가뜨리지 않았는지 확인합니다.

관련 개념 정적 분석: 타입 검사(npx tsc --noEmit), 린트(npm run lint) — 실행하지 않고 코드만 보고 오류를 찾음

단위 테스트: 함수 하나(예: stockStatus, canCarry)를 검사

통합 테스트: 서비스 + 실제 DB(예: 동시 출고 2건에서 재고가 음수가 되지 않는지)

E2E 테스트: 브라우저로 실제 화면을 조작

회귀 테스트: 고친 뒤 예전 기능이 그대로인지 다시 확인

불변식 검사: Product.stock == 이력 합계 같은 규칙을 SQL로 점검

성능 검증: 대량 데이터에서 응답 시간 측정(재고현황 3,000개 상품, 288개 조합, 최대 65ms)

이 프로젝트 **적용** 타입 검사·린트·결과 비교 검증 **미적용** 자동 테스트 도구(테스트 스크립트가 아직 없음)

⑥ 배포 · 운영 Deployment / Operation

하는 일 실제 사용자가 쓰는 서버에 올리고 안정적으로 유지합니다.

관련 개념 빌드(npm run build) · 운영 마이그레이션(prisma migrate deploy) · 비밀번호 관리(시드 기본 비밀번호 변경 필수) · HTTPS · 백업과 복구 · 로그·모니터링(오류 알림) · CI/CD(커밋하면 자동으로 검사·배포)

이 프로젝트 **부분** 운영 secure 쿠키, 감사 로그, 운영 메모 **미적용** 배포 환경, DB 백업, CI

⑦ 유지보수 Maintenance

하는 일 버그 수정, 기능 추가, 데이터가 늘었을 때의 성능 개선.

관련 개념 버전 관리(Git 커밋·브랜치) · 하위 호환(예: `/orders` 옛 주소를 새 주소로 리다이렉트, `modules/inbound/datetime.ts` 재수출) · 데이터 이관(기존 재고를 칸별 재고로 옮긴 마이그레이션) · 기술 부채 기록(“상품 많아지면 검색형으로 바꿀 것”) · 리팩터링(동작은 그대로 두고 구조 개선)

3. 요청 하나가 처리되는 흐름

예: 관리자가 출고를 확정할 때, 눈에 보이지 않는 곳에서 일어나는 일입니다.

① 브라우저 — 확정 버튼 클릭 → Server Action 호출 (품 값 + version)



② `proxy.ts` — 세션 쿠키가 없으면 `/login` 으로 (1차 확인)



③ `actions.ts` — `authorize("outbound.manage")` 권한 확인 (인가)



④ `validation.ts` — 숫자·범위·형식 서버 검증 (클라이언트 값을 믿지 않음)



⑤ `service.ts` — 트랜잭션 시작

- 상태 `version` 조건부 UPDATE (낙관적 잠금: 다른 사람이 먼저 바꿨으면 실패)
- `changeStock(tx, ...)`: 상품 행 FOR UPDATE(비관적 잠금) → 재고 확인 → 조건부 차감 → 칸별 재고 차감 → `StockMovement` 이력
- `recordAudit(tx, ...)`: 감사 로그를 같은 트랜잭션에 기록



⑥ 커밋 또는 롤백 — 중간에 하나라도 실패하면 전부 취소 (원자성)



⑦ 결과 반환 — 성공 / 한국어 오류 메시지 (예: 재고가 부족합니다. (현재고 3, 필요 5)) → `revalidatePath` 로 화면 갱신

4. 코드 구조 (모듈러 모놀리스)

warehouse-linker/	
├─ app/<screen>/	화면(페이지 · 화면 전용 컴포넌트)
├─ components/	공용 UI (Modal, Header, KpiCard, EmptyState, Skeleton, ui/*)
├─ lib/	공통 유틸 (datetime: KST, request-id, doc-number, form, prisma)
├─ modules/<domain>/	도메인별 로직
│ └─ validation.ts	서버 입력 검증
│ └─ service.ts	DB · 트랜잭션 (업무 규칙의 핵심)
│ └─ actions.ts	Server Actions (권한 확인 → 검증 → 서비스)
│ └─ codes.ts	라벨 · 상수 · 상태 색
├─ prisma/	스키마 · 마이그레이션 · 시드
├─ proxy.ts	로그인 여부 1차 확인
└─ docs/	설계 문서 · 디자인 시스템

도메인: product · inbound · outbound · stock · warehouse · order · dispatch · user · audit · dashboard

핵심 의존 방향: 화면 → actions → service → DB. 화면이 DB를 직접 건드리지 않고, 다른 도메인의 재고를 바꿀 때도 반드시 stock/service.ts 의 changeStock 을 거칩니다. 이렇게 입구를 하나로 두면 규칙이 한곳에만 있어서 빠뜨릴 수 없습니다.

2부. 핵심 개념

각 개념은 뜻 → 왜 필요한가 → 이 프로젝트에서 순서로 정리했습니다. 왼쪽 띠가 초록색인 카드 **추가**는 이번에 새로 더한 개념입니다.

1. 데이터 모델링

엔티티와 관계 (ERD) Entity-Relationship **추가**

- 뜻** 저장할 대상(엔티티)과 그 사이의 관계(1:1, 1:N, N:M)를 그림·표로 정리한 것.
- 왜** 관계를 잘못 잡으면 나중에 고치기가 가장 어렵습니다. 화면보다 먼저 정해야 합니다.
- 이** 주문(TradeOrder) 1 : N 주문 품목(TradeOrderLine), 창고 1 : N 랙 1 : N 위치(Location),
- 프로젝트** 배차 1 : N 배차 품목(출고 1건은 배차 1곳 = 1:1 제약)

정규화와 비정규화 Normalization / Denormalization **추가**

- 뜻** 정규화 = 같은 정보를 한곳에만 저장. 비정규화 = 조회 속도를 위해 계산 결과를 일부러 따로 저장.
- 왜** 중복 저장하면 두 값이 어긋날 위험이 생기므로, 비정규화를 할 때는 항상 같이 갱신하는 장치가 필요합니다.
- 이** `Product.stock`은 이력 합계를 미리 계산해 둔 값(비정규화)입니다. 어긋나지 않도록
- 프로젝트** `changeStock` 한 곳에서만 바꾸고, 불변식 `stock == Σ StockMovement`로 점검합니다.

기본키 · 외래키 · 참조 무결성 PK / FK / Referential Integrity **추가**

- 뜻** 외래키는 다른 테이블 행을 가리키는 열. 참조 무결성은 "가리키는 대상이 반드시 존재한다"는 보장.
- 왜** 대상이 지워졌을 때 어떻게 할지(`onDelete`: 함께 삭제 / NULL로 / 삭제 거부)를 정해야 고아 데이터가 생기지 않습니다.
- 이** `Product.locationId`는 `onDelete SetNull`(칸이 사라지면 미배정). 감사 로그의 `actorId`는
- 프로젝트** 일부러 FK를 두지 않음(사용자가 바뀌어도 기록은 남아야 함).

대리키와 자연키 Surrogate Key / Natural Key **추가**

- 뜻** 대리키 = 의미 없는 내부 id. 자연키 = 업무에서 쓰는 식별값(상품코드, 차량번호).
- 이** 관계는 내부 id로 연결하고, 업무 식별값(`sku`, `plateNo`, `loginId`, `orderNo`, 위치코드)은
- 프로젝트** unique 제약으로 중복만 막습니다. 업무 코드가 바뀌어도 관계가 깨지지 않습니다.

열거형과 상태 컬럼 Enum 추가

- 뜻** 허용된 값 목록을 DB 타입으로 정해 둠.
- 이** UserRole(STAFF/ADMIN), 입출고 status(PENDING/CONFIRMED/CANCELLED),
- 프로젝트** StorageType(냉장/냉동/실온), 재고이력 type 9종. 화면 라벨은 `codes.ts` 에서 한국어로 변환.

수정 이력 테이블 (변경 전후 기록) Revision / History Table 추가

- 뜻** 원본 행은 최신 값으로 두고, 바뀐 내용은 별도 테이블에 쌓음.
- 이** `InboundRevision / OutboundRevision` (수정 사유 필수, 바뀐 항목만 before/after JSON),
- 프로젝트** `ProductLocationHistory` (위치 변경 이력).

2. 데이터 정합성 — 데이터가 틀어지지 않게

트랜잭션 Transaction

- 뜻** 여러 DB 작업을 하나로 묶어 "전부 성공 아니면 전부 취소(롤백)"로 처리.
- 왜** 출고 기록은 저장됐는데 재고는 안 빠지는 식의 반쪽 상태를 막습니다.
- 이** 수주 출고 처리 중 한 품목이라도 재고가 부족하면 전체 롤백. `changeStock(tx, ...)` 은
- 프로젝트** 반드시 호출자의 트랜잭션(`tx`) 안에서 실행.

ACID Atomicity · Consistency · Isolation · Durability 추가

- 뜻** 트랜잭션이 지켜야 할 4가지 성질.
원자성: 전부 또는 전부 · 일관성: 규칙(제약)을 항상 만족 · 격리성: 동시에 실행돼도 서로 간섭 최소 · 지속성: 커밋된 결과는 사라지지 않음
- 이** 감사 로그를 작업과 같은 트랜잭션에 기록 → 작업이 롤백되면 로그도 없고, 로그 저장이
- 프로젝트** 실패하면 작업도 취소(원자성).

격리 수준 Isolation Level 추가

- 뜻** 동시에 실행되는 트랜잭션이 서로의 변경을 얼마나 볼 수 있는지의 단계. PostgreSQL 기본값은 Read Committed(커밋된 데이터만 보임).
- 왜** 기본 수준에서는 "읽고 확인한 뒤 쓰는" 사이에 다른 요청이 끼어들 수 있습니다. 그래서 아래의 잠금·조건부 UPDATE로 보완합니다.
- 이** 격리 수준을 올리는 대신 **행 잠금 + 조건부 UPDATE + unique 제약**으로 필요한 곳만
- 프로젝트** 정확히 보호.

불변식 Invariant

뜻 어떤 작업 후에도 항상 참이어야 하는 규칙.

왜 불변식이 있으면 "데이터가 맞는지"를 기계적으로 검사할 수 있고, 새 기능을 만들 때 지켜야 할 기준이 됩니다.

이 `Product.stock == ∑ StockMovement.quantity`, 항상 ≥ 0

프로젝트 `Product.stock == ∑ StockBalance.quantity` (칸별 재고 합)

- 칸마다 같은 칸 이력 합 == 칸 수량

- `processedQty == 연결된, 취소되지 않은 입고 출고 수량 합`

DB 제약조건 CHECK / UNIQUE / NOT NULL

뜻 DB가 직접 잘못된 값을 거부하는 규칙. 앱에 버그가 있어도 지켜지는 마지막 방어선.

이 `stock >= 0`, `quantity > 0`, `boxQty >= 1`, `processedQty` 0~주문수량, 취소 상태면 취소

프로젝트 사유 필수(조건부 CHECK), loginId 형식 정규식 CHECK, 한 칸에 한 상품(unique), 출고 1건은 배차 1곳(unique). Prisma 스키마로 표현할 수 없는 CHECK는 마이그레이션 SQL에 직접 추가.

NULLS NOT DISTINCT

뜻 기본적으로 unique 검사는 NULL끼리를 서로 다른 값으로 봅니다. 이 옵션(PostgreSQL 15+)은 NULL끼리도 같은 값으로 봅니다.

이 칸별 재고 unique(상품, 위치, 유통기한)에서 "위치 미지정·유통기한 미상" 칸이 중복으로

프로젝트 여러 개 생기지 않게 함.

원장(이력) 기반 재고 Ledger

뜻 현재 값만 저장하지 않고 모든 변경을 ±수량으로 쌓는 방식. 회계 장부와 같은 원리 (이벤트 소싱과 비슷한 성격).

왜 "왜 재고가 이 숫자인가"를 언제든지 추적·검증할 수 있습니다.

이 `StockMovement`: type, ±수량, 변경 전/후 재고, 관련 입고·출고, 변동된 칸. 재고현황의

프로젝트 재고 원장 팝업.

역분개 · 무효화 Reversal

뜻 기록을 지우지 않고 반대 방향 기록을 추가해 되돌림.

이 입고 취소 = 상태 CANCELLED + `INBOUND_CANCEL` (-수량). 출고 취소 =

프로젝트 `OUTBOUND_CANCEL` (+수량). 수량 정정 = `*_CORRECTION`.

추가 전용 로그 Append-only

뜻 쓰기만 되고 수정·삭제는 안 되는 저장소.

이 감사 로그에 DB 트리거(AuditLog_no_update_delete, AuditLog_no_truncate)를 걸어

프로젝트 UPDATE·DELETE·TRUNCATE를 차단. 앱에도 수정·삭제 기능 없음.

3. 동시성 — 여러 사람이 동시에 작업할 때

역등성 Idempotency

뜻 같은 요청이 여러 번 와도 결과가 한 번 처리한 것과 같은 성질.

왜 저장 버튼 두 번 클릭, 느린 네트워크로 인한 재전송 때 입고가 2건 생기는 것을 막습니다.

이 품은 저장에 성공할 때까지 같은 requestId(역등 키)를 재사용 → 서버는 이미 있는 키면

프로젝트 거부, 동시에 도착하면 unique 제약(P2002)으로 1건만 저장. 수주 처리는

requestId:lineId로 품목별 이중 처리 차단.

시나리오: 직원이 [등록]을 빠르게 두 번 클릭

요청 A (requestId=abc) —

요청 B (requestId=abc) —→ INSERT ... requestId UNIQUE

결과: A 저장 성공, B는 unique 위반 → "이미 처리된 요청입니다"

낙관적 잠금 Optimistic Lock

뜻 미리 잠그지 않고, 저장할 때 "내가 본 버전이 아직 최신인지" 확인. 충돌이 드물 때 적합.

왜 두 관리자가 같은 입고를 동시에 수정하면 나중에 저장한 쪽이 앞의 수정을 모르고 덮어씹니다(갱신 손실).

이 수정 계열은 version 필드 사용.

프로젝트

UPDATE "Inbound" SET quantity = 12, version = version + 1

WHERE id = 7 AND version = 3; — 내가 본 버전이 3일 때만

— 바뀐 행이 0건이면 → "다른 사용자가 먼저 수정했습니다"

비관적 잠금 Pessimistic Lock · SELECT ... FOR UPDATE

뜻 작업할 행을 먼저 잠가 다른 트랜잭션이 기다리게 함. 충돌이 잦거나 반드시 순서대로 처리해야 할 때.

이 재고 변경 시 상품 행, 주문 처리 시 주문 행, 락 추가 시 창고 행을 FOR UPDATE로 잠금.

프로젝트

Advisory Lock (권고 잠금)

- 뜻** 특정 행이 아니라 "작업 이름(숫자 키)"에 거는 PostgreSQL 잠금. 잠글 행이 없거나 여러 행에 걸친 작업을 한 줄로 세울 때.
- 이** 문서번호 발급(점두어+날짜별), 위치 자동 배정, 배차 배정(dispatch-assign), 활성화
- 프로젝트** 관리자 최소 1명 유지, 아이디별 로그인 실패 집계. pg_advisory_xact_lock 은 트랜잭션이 끝나면 자동으로 풀림.

조건부 UPDATE Compare-and-Set

- 뜻** 조건이 맞을 때만 값을 바꾸고, 바뀐 행 수로 성공 여부를 판단.
- 이** 재고 차감, 상태 전환(대기 → 확정), 주문 처리수량 조정.
- 프로젝트**

```
UPDATE "Product" SET stock = stock - 5
WHERE id = 42 AND stock >= 5;  -- 동시에 와도 음수가 될 수 없음
-- 0건이면 InsufficientStockError(현재고, 필요량)
```

경쟁 조건 Race Condition

- 뜻** "확인"과 "저장" 사이에 다른 요청이 끼어들어 결과가 실행 순서에 따라 달라지는 문제 (TOCTOU: Time Of Check to Time Of Use).
- 이** 사전 중복 확인을 하더라도 저장 시 unique 위반(P2002)을 다시 잡아서 처리
- 프로젝트** (isUniqueViolation). 화면에 보이던 락 수(expectedRackCount)와 실재가 다르면 거부.

교착 상태와 잠금 순서 Deadlock / Lock Ordering 추가

- 뜻** A는 1번을 잡고 2번을, B는 2번을 잡고 1번을 기다려 둘 다 영원히 멈추는 상태. DB가 감지해 한쪽을 실패시킵니다.
- 예방** 잠금을 항상 같은 순서로 잡기, 트랜잭션을 짧게 유지하기, 잠그기 전에 무거운 작업 하지 않기.
- 이** changeStock 은 항상 "상품 행 잠금 → 확인 → 총재고 → 개별 재고·이력" 순서로 고정.
- 프로젝트** 대량 작업(랙 추가)은 트랜잭션 timeout 30초 지정.

재고 예약을 하지 않는 설계 No Reservation

- 뜻** 등록 시점에 재고를 잡아두지 않고, 확정 시점에 다시 확인.
- 이** 직원의 대기 출고는 재고를 차감하지 않음 → 관리자가 확정할 때 재고가 부족하면 거부
- 프로젝트** (대기 상태 유지). 단순하지만 "등록했는데 확정 못 함"이 생길 수 있다는 트레이드오프.

4. 업무 흐름 모델링

상태 기계 State Machine

- 뜻** 정해진 상태끼리, 정해진 방향으로만 이동하게 만든 모델.
- 이** 입출고: 대기 → 확정 → 취소 (대기 건만 삭제 가능)
- 프로젝트** 주문: OPEN → PARTIAL → DONE / CLOSED(잔량 종결) / CANCELLED(처리 0건일 때만)
- 배차: 배차 → 상차 완료 → 배송중 → 배송 완료 (한 단계씩, 취소는 출발 전만)

승인 워크플로 Maker-Checker / 4-eyes

- 뜻** 입력한 사람과 승인하는 사람을 분리해 실수·부정을 줄임.
- 이** 직원이 상품·입고·출고를 "대기"로 등록 → 관리자가 확정(판매가·단가 입력) 또는 반려.
- 프로젝트** 대기 건은 재고에 반영되지 않음.

소프트 삭제와 하드 삭제 Soft / Hard Delete

- 뜻** 소프트 = 상태만 바꿔 기록 유지, 하드 = 실제로 지움.
- 이** 확정 건은 취소(소프트), 재고 이력이 없는 대기 건만 실제 삭제(하드). 차량은 삭제 대신
- 프로젝트** 운행 중지.

부분 처리 Partial Fulfillment

- 이** 주문 수량을 여러 번에 나눠 입출고. `processedQty`와 남은 수량, 잔량 종결(close)로
- 프로젝트** 마무리.

재고 버킷과 차감 순서 Bucket / FEFO

- 뜻** 재고를 상품 × 위치 × 유통기한 단위로 쪼개 관리. FEFO(First Expired, First Out) = 유통기한이 빠른 것부터 출고.
- 이** `StockBalance`. 자동 차감 순서: 유통기한 미상 → 유통기한 빠른 순 → 기본 보관위치
- 프로젝트** 먼저. 출고 위치를 직접 지정하면 그 칸에서만(strict) 차감.

되돌리기 추적 Reversal Tracing

- 이** 출고 취소·수량 감소 시 그 출고가 났던 칸으로, 나중에 뺀 칸부터 복원. 입고 취소 시 그
- 프로젝트** 입고가 넣은 칸에서 먼저 차감. 호출하는 쪽은 신경 쓰지 않아도 `changeStock`이 처리.

채번 Document Numbering

- 이** `P0-/S0-/DSP-YYYYMMDD-NNN`. 접두어+KST 날짜별 advisory lock으로 같은 번호가 두 번
- 프로젝트** 나오지 않게 함(`lib/doc-number.ts`).

유사 건 경고와 2단계 확인 Soft Validation / Confirm Step

뜻 막지는 않되 사용자에게 한 번 더 확인받는 검증. 서버가 `status: "confirm"` 을 돌려주고 저장하지 않음.

이 10분 안에 같은 상품·거래처·수량 → [그래도 등록]. 위치 변경 시 다른 상품과 교환·분류

프로젝트 불일치 경고 → `expectedOccupantId` 가 그대로일 때만 진행.

규칙 기반 호환성 검사

이 온도 등급 냉동(2) ≥ 냉장(1) ≥ 실온(0). 차량 등급 ≥ 상품 등급일 때만 적재(`canCarry`).

프로젝트 분류 → 보관 유형 매핑(`CATEGORY_STORAGE`).

5. 보안

인증과 인가 Authentication / Authorization · RBAC

뜻 인증 = 누구인지 확인(로그인). 인가 = 그 사람이 무엇을 할 수 있는지 확인. RBAC = 역할(Role)에 권한을 묶는 방식.

이 권한표 `PERMISSIONS` (`modules/user/codes.ts`)를 화면과 서버가 함께 사용. 직원: 대기

프로젝트 등록·조회, 관리자: 확정·수정·취소·관리 기능.

최소 권한 원칙 Least Privilege 추가

뜻 일에 꼭 필요한 권한만 줌.

이 직원은 금액(판매가·단가·재고금액)을 볼 권한이 없고, 재고를 바로 바꾸지 못함(대기

프로젝트 등록만).

심층 방어 Defense in Depth

뜻 한 겹이 뚫려도 다음 겹이 막도록 여러 단계로 검사.

이 `proxy.ts` (로그인 여부) → 페이지 `requirePageUser()` + `can()` → Server Action

프로젝트 `authorize()` → 서버 검증 → DB 제약. 메뉴 숨김(UI)에만 의존하지 않음 — Server Action은 주소만 알면 직접 호출될 수 있기 때문.

서버 측 입력 검증 Server-side Validation

이 모든 입력을 `validation.ts` 에서 다시 검사(정수·범위·형식·존재 여부). 브라우저 검사는

프로젝트 편의일 뿐 보안이 아님. 정렬·필터 값은 허용 목록(화이트리스트)만 받음.

데이터 최소 노출

이 금액을 CSS로 숨기는 것이 아니라 서버에서 아예 값을 내려보내지 않음(브라우저 개발자

프로젝트 도구로도 볼 수 없음).

비밀번호 해시 · 토큰 해시 저장

뜻 해시 = 되돌릴 수 없는 변환. `bcrypt`는 일부러 느리게 만든 비밀번호 전용 해시(대입 공격 비용 증가).

이 프로젝트 비밀번호는 `bcrypt` 해시로, 세션은 토큰 원문이 아닌 SHA-256 해시로 저장 → DB가 유출돼도 바로 로그인할 수 없음.

타이밍 공격 · 계정 열거 방지 Timing Attack / User Enumeration

이 프로젝트 없는 아이디도 더미 해시와 비교해 응답 시간을 같게 하고, 실패 사유를 구분하지 않으며, 없는 아이디도 똑같이 실패 횟수를 세고 잠금 → 아이디 존재 여부를 알아낼 수 없음.

무차별 대입 방지 Brute-force Protection / Throttling

이 프로젝트 같은 아이디 5회 연속 실패 → 15분 잠금(`LoginThrottle`). 잠금 중엔 비밀번호를 비교하지도 않음. 잠기는 순간 감사 로그. (IP 기준 제한은 아직 없음)

세션 관리 · 쿠키 보안 속성

뜻 `httpOnly`: 자바스크립트가 쿠키를 못 읽음(XSS 대비) · `sameSite=lax`: 다른 사이트에서 보낸 요청에 쿠키가 실리지 않음(CSRF 대비) · `secure`: HTTPS에서만 전송 · 만료: 12시간

이 프로젝트 역할 변경·사용 중지·비밀번호 재설정 시 해당 사용자의 세션 삭제(세션 무효화). 본인 비밀번호 변경 시 현재 세션만 남김.

주요 웹 공격 유형 SQL Injection · XSS · CSRF 추가

뜻 SQL 인젝션: 입력값에 SQL을 섞어 DB 조작 → Prisma와 `$queryRaw` 템플릿 문법이 값을 자동으로 분리(파라미터 바인딩)

XSS: 입력값에 스크립트를 넣어 다른 사용자 브라우저에서 실행 → React가 화면 출력 시 자동 이스케이프

CSRF: 로그인한 사용자 몰래 다른 사이트에서 요청 → `sameSite` 쿠키, Next.js Server Action의 출처(Origin) 검사

감사 추적 Audit Trail

이 프로젝트 `AuditLog`: 누가(이름·아이디), 언제, 어떤 대상에, 무엇을(변경 전후 값). 쓰기 작업을 새로 만들면 반드시 기록 추가. 화면 `/logs`.

6. 오류 처리 추가

도메인 예외 클래스 Domain Error

- 뜻** 업무 규칙 위반을 의미가 담긴 오류 타입으로 표현.
- 이** `InsufficientStockError`(현재고, 필요량), `BucketStockError`, `InboundConflictError`,
프로젝트 `DuplicateInboundRequestError`, `DuplicateSkuError`, `OrderError`, `DispatchError` 등.
서비스는 예외를 던지고, 액션이 잡아서 사용자 메시지로 바꿈.

사용자용 메시지와 개발자용 메시지 분리

- 이** 사용자에게는 구체적인 한국어 문구("재고가 부족합니다. (현재고 3, 필요 5)"), 개발자용
프로젝트 세부 오류는 서버 로그로. 내부 정보(SQL, 스택)를 화면에 노출하지 않음.

DB 오류 코드 처리 Prisma P2002 등

- 뜻** P2002 = unique 제약 위반. DB가 보내는 오류를 업무 의미로 번역.
- 이** `isUniqueViolation(e, field)` 로 어느 필드가 중복인지 확인해 "이미 등록된
프로젝트 상품코드입니다" 같은 메시지로 변환.

결과 객체 패턴 Result Object

- 뜻** Server Action이 예외를 그대로 던지지 않고 `{ status: "ok" | "error" | "confirm", message }` 같은 객체로 돌려줌.
- 왜** 화면이 성공·실패·확인 필요를 구분해 그에 맞는 UI(오류 문구, 확인 버튼)를 보여 줄 수 있음.

빠른 실패 Fail Fast

- 이** 재고를 바꾸기 전에 총재고와 칸 재고를 먼저 모두 확인 → 하나라도 부족하면 아무것도
프로젝트 바꾸지 않고 바로 실패.

7. 아키텍처와 성능

모듈러 모놀리스 Modular Monolith

- 뜻** 배포는 하나의 앱으로 하되, 내부는 도메인별 모듈로 경계를 나눔. 마이크로서비스보다
단순하고, 나중에 분리하기도 쉬움.
- 이** `modules/<도메인>/` 각각 validation · service · actions · codes.
프로젝트

계층 분리 · 단일 진입점 Layered Architecture / Single Entry Point

이 화면 → 액션(권한) → 검증 → 서비스(트랜잭션) → DB. 재고 변경은 오직 `changeStock`.
프로젝트 호출부는 버킷만 넘기고 되돌리기 규칙은 함수 안에 숨김(캡슐화).

ORM과 Raw SQL Object-Relational Mapping 추가

뜻 ORM(Prisma) = DB 테이블을 코드의 객체·타입으로 다룸. 타입 안전하고 편하지만, 복잡한 집계나 잠금은 직접 SQL이 필요할 때가 있음.
이 대부분 Prisma API, 재고금액 집계(`SUM(stock*price)`)·FOR UPDATE·advisory
프로젝트 lock·CHECK·트리거는 Raw SQL.

DB에서 처리하기 Push-down

뜻 전체를 앱으로 가져와 JS로 거르지 않고, 필터·정렬·합계·페이지 나누기를 SQL에서 처리.
이 재고현황 전부 DB 처리. 최근 입출고일은 현재 페이지 상품 id만 `groupBy`. 3,000개
프로젝트 상품에서 최대 65ms.

페이지네이션과 타이브레이커 Pagination / Tie-breaker

뜻 OFFSET(skip/take) 방식은 단순하지만 뒤 페이지일수록 느려짐(대안: 키셋 페이지네이션). 정렬 값이 같은 행이 있으면 순서가 흔들려 같은 행이 두 페이지에 나올 수 있음 → 고유한 보조 정렬 키 추가.
이 50건씩, sku를 타이브레이커로, 범위 밖 페이지는 마지막 페이지로 보정.
프로젝트

인덱스 Index

뜻 책의 색인처럼 특정 열로 빠르게 찾기 위한 자료구조. 조회는 빨라지고 쓰기는 약간 느려짐. 복합 인덱스는 열 순서가 중요.
이 (category, sku), (stock), 감사 로그 (createdAt), (category, createdAt). 부분 검색이
프로젝트 느려지면 `pg_trgm` 인덱스 후보.

일괄 처리 Bulk Insert

이 랙·칸 생성 시 id를 미리 만들어 `createMany` 로 한 번에 삽입(한 요청 최대 20,000칸). N번
프로젝트 왕복 대신 1번.

N+1 쿼리 문제 N+1 Query 추가

뜻 목록 1번 조회 후 각 행마다 추가 조회를 N번 하는 비효율.
이 재고현황의 최근 입출고일을 행마다 조회하지 않고 현재 페이지 id 묶음으로 한 번에
프로젝트 `groupBy`.

시간대 정규화 Time Zone

- 뜻** DB는 UTC로 저장하고, 업무 기준(하루의 시작·끝, 표시)은 한 시간대로 통일.
- 이** 모든 업무 날짜는 KST. "오늘" = KST 자정~자정, 납기일·배송일 = KST 자정. 변환은
- 프로젝트** `lib/datetime.ts` 한 곳에서.

8. Next.js · 프론트엔드 추가

서버 컴포넌트와 클라이언트 컴포넌트

- 뜻** 서버 컴포넌트 = 서버에서 HTML을 만들어 보냄(DB 직접 조회 가능, JS 용량 없음). 클라이언트 컴포넌트(`"use client"`) = 브라우저에서 동작(클릭·입력 상태).
- 이** 페이지는 서버에서 권한 확인과 조회, 폼·모달·선택 상자는 클라이언트 컴포넌트. 서버 →
- 프로젝트** 클라이언트로 넘기는 값은 직렬화 가능해야 함(날짜·Decimal 변환).

Server Actions · useActionState

- 뜻** 폼에서 서버 함수를 직접 호출하는 방식. `useActionState` 로 결과(성공·오류·확인 필요)와 처리 중 상태를 화면에 연결.
- 주의** Server Action은 공개된 엔드포인트와 같음 → 반드시 시작 부분에서 `authorize`.

캐시와 재검증 · 동적 렌더링

- 뜻** Next.js는 페이지 결과를 캐시할 수 있음. 데이터가 바뀌면 `revalidatePath` 로 다시 만들게 하고, 항상 최신이어야 하는 페이지는 `await connection()` 으로 요청마다 렌더링.
- 주의** 저장 후 재검증으로 컴포넌트가 다시 마운트되면 상태가 사라질 수 있음 → 모달 key에 `version`을 넣지 않음.

URL에 화면 상태 담기 URL State

- 이** 재고현황 필터·정렬·페이지, 배차 날짜, 주문 상태를 쿼리 문자열(`?status=low&page=2`)로
- 프로젝트** → 새로그침·북마크·뒤로가기에도 유지. GET 폼이라 자바스크립트 없이도 동작.

스트리밍과 로딩 UI · 네이티브 dialog

- 이** `app/loading.tsx` + Skeleton으로 데이터가 오기 전에 화면 틀 먼저 표시. 모달은
- 프로젝트** 브라우저 기본 `<dialog>` (`components/Modal.tsx`) — 포커스 이동·Esc 닫기 같은 접근성 기능을 기본 제공.

9. UI/UX 디자인 시스템

디자인 토큰 Design Token

뜻 색·반경·간격·글자 크기를 이름 붙은 변수로 정의해 한곳에서 관리. 값을 바꾸면 전 화면에 반영.

이 app/globals.css 의 :root — --primary(#315EEA), --sidebar*, --radius-* (입력·
프로젝트 버튼 8px, 배지 6px, 카드 12px). Tailwind 클래스(bg-primary)로 사용.

디자인 시스템 · 컴포넌트 재사용

뜻 같은 기능은 모든 화면에서 같은 모양·같은 동작. 새 스타일을 즉흥적으로 만들지 않음.

이 Indigo Enterprise. components/ui/* (shadcn/ui), Modal, KpiCard, EmptyState. 높이
프로젝트 규격: 버튼·입력 36px, 헤더 64px, 표 행 48px.

시맨틱 컬러 · 색에만 의존하지 않기 Accessibility

이 상태 → 색은 *_STATUS_TONE (정상 초록, 부족 노랑, 품질 빨강, 예약 파랑). 색맹 사용자도
프로젝트 알 수 있게 항상 라벨을 함께 표시, 증감은 ▲▼ 기호와 문구.

정보 밀도와 표 정렬 규칙

이 날짜·상태 = 가운데, 상품명·코드·거래처 = 왼쪽(.left), 수량·금액 = 오른쪽(.num),
프로젝트 자릿수 비교가 쉬움). 수량은 단위까지 표기.

빈 상태 · 스켈레톤 · 오류 문구

이 데이터가 없으면 아이콘+안내 문구(EmptyState), 로딩 중에는 화면 모양(Skeleton),
프로젝트 오류는 사용자가 이해할 수 있는 한국어.

폰트 자체 호스팅

이 Pretendard(한글)·Inter(영문·숫자)를 app/fonts/ 에서 직접 제공 — 외부 CDN 장애·사내망
프로젝트 차단에 영향받지 않음.

10. 개발 도구와 운영 추가

버전 관리 Git

뜻 코드 변경 이력을 커밋 단위로 저장. 문제가 생기면 이전 상태로 되돌리거나 원인 커밋을 찾을 수 있음.

좋은 습관 기능 하나 = 커밋 하나, 무엇을 왜 바꿨는지 커밋 메시지에 기록, 큰 작업은 브랜치에서.

마이그레이션 Schema Migration

- 뜻** DB 구조 변경을 순서가 있는 SQL 파일로 기록 → 어느 환경에서든 같은 순서로 적용.
- 이** `npx prisma migrate dev --create-only --name <이름>` 로 만든 뒤 SQL에 CHECK·
- 프로젝트** 트리거를 직접 추가. 이미 적용된 마이그레이션은 수정하지 않고 새로 추가.

컨테이너 Docker / docker compose

- 뜻** 프로그램과 실행 환경을 상자(컨테이너)로 묶어 어디서나 똑같이 실행. volume으로 데이터 보존.
- 이** PostgreSQL 15 컨테이너 `warehouse-linker_pg`. "Can't reach database server" 오류는
- 프로젝트** Docker Desktop이 꺼져 있는지 먼저 확인.

환경 변수와 비밀값 Environment Variables / Secrets

- 뜻** DB 주소·비밀번호처럼 환경마다 다르거나 공개되면 안 되는 값을 코드 밖에서 주입. Git에 올리지 않음.
- 이** 시드 기본 계정(admin/staff)과 compose의 DB 비밀번호는 개발용 — 운영 전 반드시
- 프로젝트** 변경.

정적 분석 Type Check / Lint

- 뜻** 실행하지 않고 코드를 분석해 오류를 찾음. TypeScript 타입 검사는 "없는 필드 사용, 잘못된 타입 전달"을, ESLint는 "규칙 위반, 실수하기 쉬운 패턴"을 잡음.
- 이** 기능 구현 후 `npx tsc --noEmit`, `npm run lint` 실행.
- 프로젝트**

문서화 Living Documentation

- 뜻** 코드와 함께 계속 갱신되는 문서. "무엇을"보다 "왜 이렇게 정했는지"가 중요.
- 이** 업무 규칙·데이터 모델을 바꾸면 `docs/DESIGN.md` 도 함께 갱신(예: "참고 선택은 재고가
- 프로젝트** 상품 단위라 넣지 않음 — 참고별 재고를 만들 때 추가").

백업 · 모니터링 · CI/CD 미적용

- 뜻** 백업: 정기적으로 DB를 복사하고 복구 연습 · 모니터링: 오류·응답 시간 감시와 알림 · CI/CD: 커밋하면 자동으로 타입 검사·린트·테스트 후 배포.
- 이** 운영 전에 준비할 항목. 특히 재고·감사 데이터는 되돌릴 수 없으므로 백업이 우선.
- 프로젝트**

3부. 공부 순서 추천과 용어 색인

추천 공부 순서

단계	개념	이 프로젝트에서 확인할 곳
1. 기초	관계형 DB, 기본키·외래키, SQL(SELECT/UPDATE/JOIN), 인덱스	<code>prisma/schema.prisma</code>
2. 정합성	트랜잭션, ACID, 제약조건, 불변식, 원장	<code>modules/stock/service.ts</code> , 마이그레이션 SQL
3. 동시성	경쟁 조건, 낙관적·비관적 잠금, 조건부 UPDATE, advisory lock, 교착 상태	<code>changeStock</code> , <code>lib/doc-number.ts</code>
4. 멍등성	요청키, unique 제약, 중복 방지	<code>lib/request-id.ts</code> , 입고·출고 service
5. 업무 모델	상태 기계, 승인 흐름, 소프트 삭제, 역분개	입고·출고·주문·배차 service
6. 보안	인증·인가, 해시, 세션·쿠키, 웹 공격 유형	<code>modules/user/</code> , <code>proxy.ts</code>
7. 웹 프레임워크	서버·클라이언트 컴포넌트, Server Action, 캐시·재검증	<code>app/</code> , 각 <code>actions.ts</code>
8. UI	디자인 토큰, 컴포넌트 재사용, 접근성	<code>app/globals.css</code> , <code>components/</code>
9. 운영	Git, Docker, 마이그레이션, 테스트, CI/CD, 백업	<code>docker-compose.yml</code> , <code>prisma/migrations/</code>

용어 색인 (가나다순)

용어	영문	한 줄 정의
격리 수준	Isolation Level	동시 트랜잭션이 서로의 변경을 보는 정도
경쟁 조건	Race Condition	실행 순서에 따라 결과가 달라지는 동시성 버그
교착 상태	Deadlock	서로 상대의 잠금을 기다리며 멈춘 상태
낙관적 잠금	Optimistic Lock	저장할 때 버전을 비교해 충돌 감지
디자인 토큰	Design Token	색·반경 등 디자인 값을 변수로 관리
롤백	Rollback	트랜잭션의 모든 변경을 취소
마이그레이션	Migration	DB 구조 변경을 순서 있는 파일로 관리
멱등성	Idempotency	여러 번 실행해도 한 번 실행한 것과 같은 결과
모듈러 모놀리스	Modular Monolith	하나로 배포하되 내부를 도메인별로 분리
불변식	Invariant	항상 참이어야 하는 데이터 규칙
비관적 잠금	Pessimistic Lock	작업 전에 행을 잠가 다른 작업을 대기시킴
비정규화	Denormalization	조회 성능을 위해 계산 결과를 따로 저장
상태 기계	State Machine	정해진 상태 간 정해진 이동만 허용
소프트 삭제	Soft Delete	지우지 않고 상태만 바꿔 기록 유지
심층 방어	Defense in Depth	여러 겹으로 보안 검사
원장	Ledger	모든 변동을 ±로 쌓는 이력 장부
원자성	Atomicity	전부 성공 아니면 전부 취소
인가	Authorization	무엇을 할 수 있는지 확인
인증	Authentication	누구인지 확인
조건부 UPDATE	Compare-and-Set	조건이 맞을 때만 값을 바꿈
참조 무결성	Referential Integrity	외래키가 가리키는 대상이 반드시 존재
채번	Numbering	문서 번호를 중복 없이 발급
타이브레이커	Tie-breaker	정렬 값이 같을 때 순서를 정하는 보조 키
트랜잭션	Transaction	여러 DB 작업을 하나의 단위로 묶음
해시	Hash	되돌릴 수 없는 변환(비밀번호·토큰 저장)
Advisory Lock	—	행이 아닌 작업 이름에 거는 PostgreSQL 잠금
FEFO	First Expired, First Out	유통기한 빠른 것부터 출고

RBAC	Role-Based Access Control	역할 단위로 권한 부여
------	---------------------------	--------------